CHAPTER 4

What Makes Great Tests

As the Rails community has matured, Rails developers have become much more likely to work with codebases and test suites that contain many years' worth of work. As a result, there has been a lot of discussion about design strategies to manage complexity over time.

There hasn't been nearly as much discussion about what practices make tests and test suites continue to be valuable over time. As applications grow, as suite runs get longer, as complexity increases, how can you write tests that will be useful and not impede future development?

In this chapter I'll discuss various ways you can evaluate whether you're writing tests that provide value and that minimize cost.

The Big One

This is the best general piece of advice I can give about the style and structure of automated tests: Your code is verified by your tests, but your tests are verified by nothing. Having your tests be as clear and as manageable as possible is the best way to keep them honest.

> **Prescription 10** Your tests are also code. Specifically, your tests are code that does not have tests.

In practice this means avoiding complex metaprogramming or loops in test code that might make it challenging to understand and evaluate test failure. One thing you might be tempted to do from time to time is invoke RSpec it methods inside a loop. This is a bad idea. Even though handling very similar logic in separate tests seems like more code, giving each test a unique line of code for failure reporting makes it much easier to determine what's going on and much easier to isolate in individual test failure.